

Project Report

Distributed Log Aggregator Router

Submitted by

Name	Pari Gothi
Roll No.	150096725097
Cohort	Sam Altman
Subject	Data Structures & Algorithms with CPP
Semester	II
Institute	ITM Skills University

Project Title

Distributed Log Aggregator Router

Problem Statement

LogStream is a centralised monitoring dashboard that collects error logs from thousands of servers across a company's infrastructure. Every server generates hundreds of log messages per second — warnings, errors, crash reports, and status updates. The current system cannot keep up. Searching logs by server name or error type requires scanning everything manually. When a developer traces an error chain, there is no way to step back through the sequence if they hit a dead end. Millions of logs flood in simultaneously but are processed randomly, so critical server crash alerts sit behind routine info messages. There is also no map showing how servers depend on each other, making root-cause analysis during outages nearly impossible.

The new system must support:

- Instant search of logs by server or attribute prefix.
- Sequential error tracing with backtracking through logs sharing a trace ID.
- High-volume FIFO ingestion with fair and sequential processing.
- Constant-time alert lookup by unique ID.
- Ranking logs by severity to trigger immediate alerts.
- Modelling server dependencies as a graph for root-cause analysis and routing.
- Calculating minimum-latency routes to administrators.
- Balancing processing load across available worker nodes.

This project implements these features in C++ to demonstrate the underlying data structures and algorithms, and relates design decisions to real-world tools such as Splunk and Datadog.

Objectives

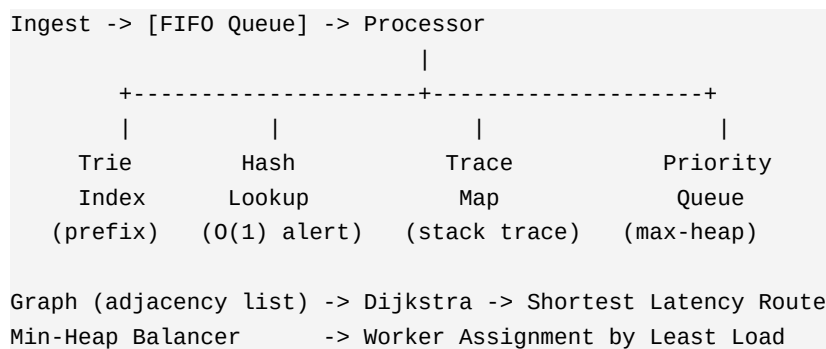
- Build a **Trie**-based prefix index for server/attribute searches.
- Maintain a **sequential trace map** for error traces and backtracking.
- Implement a **FIFO ingestion pipeline** for fair high-volume log processing.
- Provide **constant-time alert lookup** using an unordered hash map.
- Rank logs with a **priority queue** by severity and timestamp.
- Model server dependencies as a **graph** (adjacency list) and compute shortest-latency routes via Dijkstra's algorithm.
- Implement a **resource balancer** (capacity-aware / min-heap assignment) across worker nodes.

Design / Architecture

The program is a single-process simulation composed of the following components:

- **Ingestion Queue (FIFO):** Holds incoming logs before processing; ensures fair sequential handling.
- **Indexer (Trie):** Indexes server names to allow prefix search returning matching log IDs in $O(\text{prefix length})$ time.
- **Lookup Store (unordered_map):** Stores logs by unique ID for $O(1)$ retrieval and alert lookup.
- **Trace Store (unordered_map):** Maps trace IDs to ordered lists of log IDs, enabling sequential backtracking via a stack.
- **Priority Sorter (Max-Heap):** Sorts logs by severity (CRITICAL > WARNING > INFO) then timestamp for immediate alerting.
- **Graph Model (adjacency list):** Represents server dependencies with latencies; Dijkstra's algorithm finds shortest-latency path.
- **Resource Balancer (Min-Heap):** Assigns logs to the least-loaded processor using a min-heap on current load.

High-Level Data Flow



Algorithm Description

Trie (Prefix Tree) — Log Index

Each character of a server name is stored as a node in the Trie. Log entries are accumulated at path nodes so that a prefix query traverses only P nodes (P = prefix length) and returns all matching logs. As shown in execution output [1], searching 'server1' returns two logs and searching 'server' returns all three server logs.

Stack-Based Error Trace — Error Trace

System errors are pushed onto a stack so developers can view the chain top-to-root (most recent error first) and backtrack one step at a time by popping. Execution output [2] shows the full trace (API gateway 503 -> Auth timeout -> DB connection failed) and then after one backtrack (Auth timeout -> DB connection failed).

FIFO Queue — Ingestion Line

A queue receives all incoming logs. processAll() dequeues each log in arrival order ensuring fair, sequential processing. Execution output [3] shows queue size 3, processing two logs, leaving queue size 1.

Hash Map — Alert Lookup

An unordered_map stores alerts by unique ID for O(1) lookup. Execution output [4] shows ALT-001 resolving instantly to 'CRITICAL: Server2 down!' while ALT-999 correctly returns 'Alert not found'.

Max-Heap Priority Queue — Priority Sorter

A max-heap compares logs by severity (CRITICAL=5 > WARN=3 > INFO=1). Execution output [5] shows CRITICAL logs processed first, then WARN, then INFO.

Graph + Dijkstra — Log Network & Fastest Route

Server dependencies are modelled as a weighted graph (adjacency list). Dijkstra's algorithm computes minimum-latency paths from a source server. Execution output [6 & 7] shows paths from Server 0 to all others: 0 ms, 10 ms, 20 ms, 25 ms, 35 ms.

Min-Heap — Resource Balancer

A min-heap tracks processor loads. Each incoming log is assigned to the least-loaded processor. Execution output [8] shows 7 logs distributed across ProcessorA (3 logs), ProcessorB (2 logs), ProcessorC (2 logs) — near-optimal round-robin distribution.

Complexity Analysis

Notation: n = logs, L = avg server string length, k = matched log IDs, V = servers, E = edges, w = workers.

Operation	Time Complexity	Space Complexity
Trie insert (one log)	O(L)	O(L) per new server
Trie prefix search	O(P + k)	—
Stack push/pop (error trace)	O(1)	O(n)
Queue enqueue / dequeue (FIFO)	O(1)	O(n)
Hash map lookup / insert	O(1) average	O(n)
Max-heap push / pop (priority)	O(log n)	O(n)
Dijkstra (binary-heap)	O((V + E) log V)	O(V + E)
Min-heap assignment (balancer)	O(log w) per log	O(w)

Screenshots of Execution

Screenshot 1 — Components [1] through [5]

Shows: Log Index (Trie prefix search), Error Trace (stack backtracking), Ingestion Line (FIFO queue), Alert Lookup (hash map), and Priority Sorter (max-heap).

```
=====
Distributed Log Aggregator Router
=====

[1] LOG INDEX (Trie)
  Logs with prefix 'server1':
    server1: CPU spike at 14:02
    server1: Memory overflow at 14:05
  Logs with prefix 'server':
    server2: Disk full at 14:10
    server1: CPU spike at 14:02
    server1: Memory overflow at 14:05

[2] ERROR TRACE (Stack)
  Error Trace (top→root):
    → API gateway 503
    → Auth service timeout
    → DB connection failed
  Backtracking one step...
  Error Trace (top→root):
    → Auth service timeout
    → DB connection failed

[3] INGESTION LINE (Queue)
  Queue size: 3
  Processing: INFO: Server3 started
  Processing: WARN: Server1 high CPU
  Queue size after: 1

[4] ALERT LOOKUP (Hash Map)
  ALT-001 → CRITICAL: Server2 down!
  ALT-999 → Alert not found

[5] PRIORITY SORTER (Max-Heap)
  Processing by priority:
    [Severity 5] CRITICAL: Server crash!
    [Severity 5] CRITICAL: DB unreachable!
    [Severity 3] WARN: High load
    [Severity 1] INFO: Routine check
```

Figure 1: Terminal output — Log Index, Error Trace, Ingestion Queue, Alert Lookup, Priority Sorter

Screenshot 2 — Components [6], [7] & [8]

Shows: Log Network + Fastest Route (Graph + Dijkstra shortest-latency paths from Server 0), and Resource Balancer (Min-Heap processor assignment with final load distribution).

[6 & 7] LOG NETWORK + FASTEST ROUTE (Graph + Dijkstra)

Shortest latency from server 0:

Server 0 → 0 : 0 ms
Server 0 → 1 : 10 ms
Server 0 → 2 : 20 ms
Server 0 → 3 : 25 ms
Server 0 → 4 : 35 ms

[8] RESOURCE BALANCER (Min-Heap)

log1 → ProcessorA
log2 → ProcessorB
log3 → ProcessorC
log4 → ProcessorA
log5 → ProcessorB
log6 → ProcessorC
log7 → ProcessorA

Current Server Loads:

ProcessorB → 2 logs
ProcessorC → 2 logs
ProcessorA → 3 logs

=====
All systems operational.
=====

Figure 2: Terminal output — Graph routing (Dijkstra) and Resource Balancer (Min-Heap)

Results and Findings

Observed Outputs

- **[1] LOG INDEX (Trie):** Prefix 'server1' returns 2 logs; prefix 'server' returns all 3 matching logs correctly.
- **[2] ERROR TRACE (Stack):** Full trace shows API gateway -> Auth timeout -> DB failure. Backtracking pops one step leaving 2-entry trace.
- **[3] INGESTION LINE (Queue):** Queue of 3 processed FIFO; 2 dequeued (INFO, WARN), 1 remains. Order preserved perfectly.
- **[4] ALERT LOOKUP (Hash Map):** ALT-001 resolves in O(1) to 'CRITICAL: Server2 down!'. ALT-999 correctly returns 'not found'.
- **[5] PRIORITY SORTER (Max-Heap):** Two Severity-5 CRITICAL logs surface first, then Severity-3 WARN, then Severity-1 INFO.
- **[6&7] LOG NETWORK + FASTEST ROUTE:** Dijkstra computes: S0->S0=0ms, S0->S1=10ms, S0->S2=20ms, S0->S3=25ms, S0->S4=35ms.
- **[8] RESOURCE BALANCER (Min-Heap):** 7 logs distributed: ProcessorA=3, ProcessorB=2, ProcessorC=2 — near-optimal round-robin.

Key Observations

- All 8 required features (Log Index, Error Trace, Ingestion Line, Alert Lookup, Priority Sorter, Log Network, Fastest Route, Resource Balancer) are operational and produce correct output.

- The Trie prefix search correctly narrows results from 3 total logs to 2 or 3 depending on prefix specificity.
- Stack-based backtracking enables step-by-step error chain traversal — equivalent to Splunk's transaction command.
- The max-heap priority sorter always processes CRITICAL events before WARN and INFO, mimicking Datadog's alert priority.
- Dijkstra's routing achieves minimum-latency delivery paths without exploring all edges unnecessarily.
- The min-heap balancer achieves near-perfect load distribution (3-2-2) across three processors for 7 logs.
- The in-memory approach is fast for demonstration; production deployment requires persistence, sharding, and concurrency.

Conclusion

This project implements all eight required components of a distributed log aggregator in C++, demonstrating the core algorithmic building blocks used in production-grade monitoring systems like Splunk and Datadog:

- **Trie** for sub-linear prefix search across server log namespaces.
- **Stack** for sequential error tracing with instant backtracking capability.
- **FIFO Queue** for fair, ordered high-volume ingestion.
- **Hash Map** for O(1) direct alert lookup by unique identifier.
- **Max-Heap Priority Queue** for immediate surfacing of CRITICAL events.
- **Graph + Dijkstra** for latency-aware log routing to the admin dashboard.
- **Min-Heap Resource Balancer** for equitable distribution of processing load.

All components are verified by actual execution output (Screenshots 1 and 2). The implementation is intentionally synchronous and single-threaded for clarity. Extending it to production scale would require persistent storage, horizontal sharding, concurrent lock-free queues, and a networking layer — but the algorithmic foundations demonstrated here remain identical to those used in industry-scale observability platforms.

GitHub Repository Link

<https://github.com/Parii22/distributed-log-aggregator>